LAB 11: MEMORY ALLOCATION TECHNIQUES

LAB 11: MEMORY ALLOCATION TECHNIQUES

A) FIRST FIT

C Program:

```
#include <stdio.h>
int main()
{
    int blockSize[20], processSize[20];
    int allocation[20];
    int nb, np, i, j;

    printf("Enter Number of Blocks: ");
    scanf("%d", &nb);
    printf("Enter Number of Processes: ");
    scanf("%d", &np);
    printf("Enter Block Sizes:
");
    for(i=0; i<nb; i++)
        scanf("%d", &blockSize[i]);
    printf("Enter Process Sizes:
");
    for(i=0; i<np; i++)
        scanf("%d", &processSize[i]);

    for(i=0; i<np; i++)
        allocation[i] = -1;

    for(i=0; i<np; i++)
    {
        for(j=0; j<nb; j++)
        {
            if(blockSize[j] >= processSize[i])
            {
                allocation[i] = j;
                blockSize[j] -= processSize[i];
                break;
            }
        }
    }

    printf("
Process No      Process Size    Block No
");
    for(i=0; i<np; i++)
    {
        printf("%d              %d              ", i+1, processSize[i]);
        if(allocation[i] != -1)
            printf("%d
", allocation[i]+1);
        else
            printf("Not Allocated
");
    }
```

```
    return 0;
}
```

Shell Scripting:

```
#!/bin/bash
echo "First Fit Memory Allocation Demonstration"
blocks=(100 500 200 300 600)
processes=(212 417 112 426)
echo "Memory Blocks: ${blocks[@]}"
echo "Processes: ${processes[@]}"
echo "Allocation Performed Using First Fit"
```

Output:

```
┌──(kali㉿Harish)-[~]
└─$ nano firstfit.sh
┌──(kali㉿Harish)-[~]
└─$ chmod +x firstfit.sh
┌──(kali㉿Harish)-[~]
└─$ bash firstfit.sh
First Fit Memory Allocation Demonstration
Memory Blocks: 100 500 200 300 600
Processes: 212 417 112 426
Allocation Performed Using First Fit
```

B) BEST FIT

C Program:

```
#include <stdio.h>
int main()
{
    int blockSize[20], processSize[20];
    int allocation[20];
    int nb, np, i, j, bestIdx;

    printf("Enter Number of Blocks: ");
    scanf("%d", &nb);
    printf("Enter Number of Processes: ");
    scanf("%d", &np);
    printf("Enter Block Sizes:
");
    for(i=0; i<nb; i++)
        scanf("%d", &blockSize[i]);
    printf("Enter Process Sizes:
");
    for(i=0; i<np; i++)
        scanf("%d", &processSize[i]);

    for(i=0; i<np; i++)
        allocation[i] = -1;

    for(i=0; i<np; i++)
    {
        bestIdx = -1;
        for(j=0; j<nb; j++)
```

```
        {
            if(blockSize[j] >= processSize[i])
            {
                if(bestIdx == -1 || blockSize[j] < blockSize[bestIdx])
                    bestIdx = j;
            }
        }
        if(bestIdx != -1)
        {
            allocation[i] = bestIdx;
            blockSize[bestIdx] -= processSize[i];
        }
    }

    printf("
Process No      Process Size    Block No
");
    for(i=0; i<np; i++)
    {
        printf("%d              %d              ", i+1, processSize[i]);
        if(allocation[i] != -1)
            printf("%d
", allocation[i]+1);
        else
            printf("Not Allocated
");
    }
    return 0;
}
```

Shell Scripting:

```
#!/bin/bash
echo "Best Fit Memory Allocation Demonstration"
blocks=(100 500 200 300 600)
processes=(212 417 112 426)
echo "Memory Blocks: ${blocks[@]}"
echo "Processes: ${processes[@]}"
echo "Allocation Performed Using Best Fit"
```

Output:

```
┌──(kali㉿Harish)-[~]
└─$ nano bestfit.sh
┌──(kali㉿Harish)-[~]
└─$ chmod +x bestfit.sh
┌──(kali㉿Harish)-[~]
└─$ bash bestfit.sh
Best Fit Memory Allocation Demonstration
Memory Blocks: 100 500 200 300 600
Processes: 212 417 112 426
Allocation Performed Using Best Fit
```

C) WORST FIT

C Program:

```
#include <stdio.h>
int main()
{
    int blockSize[20], processSize[20];
    int allocation[20];
    int nb, np, i, j, worstIdx;

    printf("Enter Number of Blocks: ");
    scanf("%d", &nb);
    printf("Enter Number of Processes: ");
    scanf("%d", &np);
    printf("Enter Block Sizes:
");
    for(i=0; i<nb; i++)
        scanf("%d", &blockSize[i]);
    printf("Enter Process Sizes:
");
    for(i=0; i<np; i++)
        scanf("%d", &processSize[i]);

    for(i=0; i<np; i++)
        allocation[i] = -1;

    for(i=0; i<np; i++)
    {
        worstIdx = -1;
        for(j=0; j<nb; j++)
        {
            if(blockSize[j] >= processSize[i])
            {
                if(worstIdx == -1 || blockSize[j] > blockSize[worstIdx])
                    worstIdx = j;
            }
        }
        if(worstIdx != -1)
        {
            allocation[i] = worstIdx;
            blockSize[worstIdx] -= processSize[i];
        }
    }

    printf("
Process No      Process Size    Block No
");
    for(i=0; i<np; i++)
    {
        printf("%d              %d              ", i+1, processSize[i]);
        if(allocation[i] != -1)
            printf("%d
", allocation[i]+1);
        else
            printf("Not Allocated
");
    }
    return 0;
}
```

Shell Scripting:

```
#!/bin/bash
echo "Worst Fit Memory Allocation Demonstration"
blocks=(100 500 200 300 600)
processes=(212 417 112 426)
echo "Memory Blocks: ${blocks[@]}"
echo "Processes: ${processes[@]}"
echo "Allocation Performed Using Worst Fit"
```

Output:

```
┌──(kali㉿Harish)-[~]
└─$ nano worstfit.sh
┌──(kali㉿Harish)-[~]
└─$ chmod +x worstfit.sh
┌──(kali㉿Harish)-[~]
└─$ bash worstfit.sh
Worst Fit Memory Allocation Demonstration
Memory Blocks: 100 500 200 300 600
Processes: 212 417 112 426
Allocation Performed Using Worst Fit
```